

Многоуровневое проектирование: от C4 Model до спецификации API

Платформа товарных знаний на базе мультимодального GraphRAG

Домашнее задание 2. Курс OTUS “ИИ-архитектор”

Содержание

Контекст	1
1. Уровень 2. Диаграмма контейнеров	2
2. Уровень 3. Компоненты сервиса агента	2
3. Диаграммы последовательности	4
Сценарий 1а. Приём запроса и извлечение фактов	4
Сценарий 1б. Генерация ответа и доставка	4
Сценарий 2. Запрос данных вне уровня доступа	5
Связность с уровнем 3	6
4. Спецификация API	6
Разграничение ответственности	6
Запрос	6
Ответ	7
Коды ответов	7
Потоковый режим	8
Документы проекта	8

Контекст

Репозиторий проекта: https://github.com/SandalAndrey/otus_ai_architect

Заказчик - интернет-магазин коллекционных масштабных моделей: более 15 000 позиций в наличии, около 400 автомобильных марок, порядка 600 производителей.

Система отвечает на товарные вопросы, извлекая факты обходом графа знаний, а не векторным поиском по тексту. Запросы вида “что ещё выпускалось по этому прототипу” или “чем отличаются два выпуска” в плоском поиске не решаются.

Три ограничения определяют все решения ниже:

1. Закрытый контур. Обращения к внешним LLM-API исключены архитектурно, поэтому сервис инференса находится внутри системы, а не за её границей.
2. Разграничение доступа на уровне узлов графа. Отсутствие утечки данных ограниченного доступа - блокирующий критерий приёмки.
3. На этапе разработки доступен только CPU, поэтому тяжёлая обработка документов вынесена в офлайн-конвейер.

Модель C4 описана кодом на LikeC4, все диаграммы ниже построены из одного исходника [docs/diagrams/model.c4](#).

1. Уровень 2. Диаграмма контейнеров

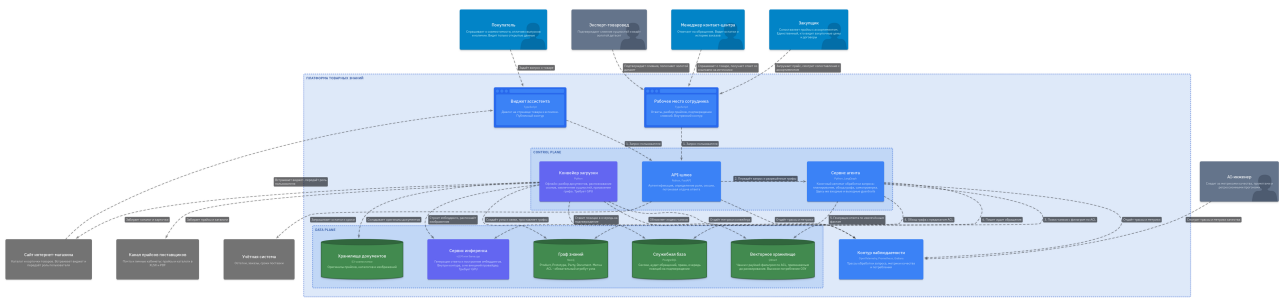


Рис. 1: Уровень 2. Контейнеры

Контейнеры разделены на две плоскости, и это не оформительский приём.

Control Plane - то, что принимает решения: API-шлюз, сервис агента, конвейер загрузки.
Data Plane - то, что хранит и считает: сервис инференса, граф знаний, векторное хранилище, служебная база, хранилище документов.

Разделение нужно, чтобы масштабировать эти части независимо. Нагрузка на них разной природы: агент упирается в задержки обращений к хранилищам, а инференс - в видеопамять и пропускную способность.

Контейнер	Технология	Назначение
Виджет ассистента	TypeScript	Диалог на странице товара, публичный контур
Рабочее место сотрудника	TypeScript	Ответы, разбор прайсов, подтверждение слияний
API-шлюз	Python, FastAPI	Аутентификация, определение роли, сессии, потоковая отдача
Сервис агента	Python, LangGraph	Конечный автомат обработки запроса
Конвейер загрузки	Python	Офлайн: разбор документов, распознавание, извлечение сущностей
Сервис инференса	vLLM или llama.cpp	Генерация и эмбединги, требует GPU
Граф знаний	Neo4j	Product, Prototype, Party, Document с метками доступа
Векторное хранилище	Qdrant	Чанки с фильтром по метке, высокое потребление ОЗУ
Служебная база	PostgreSQL	Сессии, аудит, права, очередь на подтверждение
Хранилище документов	S3-совместимое	Оригиналы прайсов и изображений
Контур наблюдаемости	OpenTelemetry, Prometheus	Трассы и метрики

Внешнего поставщика LLM на диаграмме нет намеренно. В типовой схеме RAG там стоит облачный провайдер, и туда же выносят фильтрацию персональных данных. У нас модель внутри контура, поэтому фильтрация решает другую задачу: не выпустить наружу закрытые данные в ответе пользователю.

2. Уровень 3. Компоненты сервиса агента

Раскрыт именно этот контейнер: он лежит на пути пользовательского сценария, поэтому шаги диаграмм последовательности ссылаются на его компоненты.

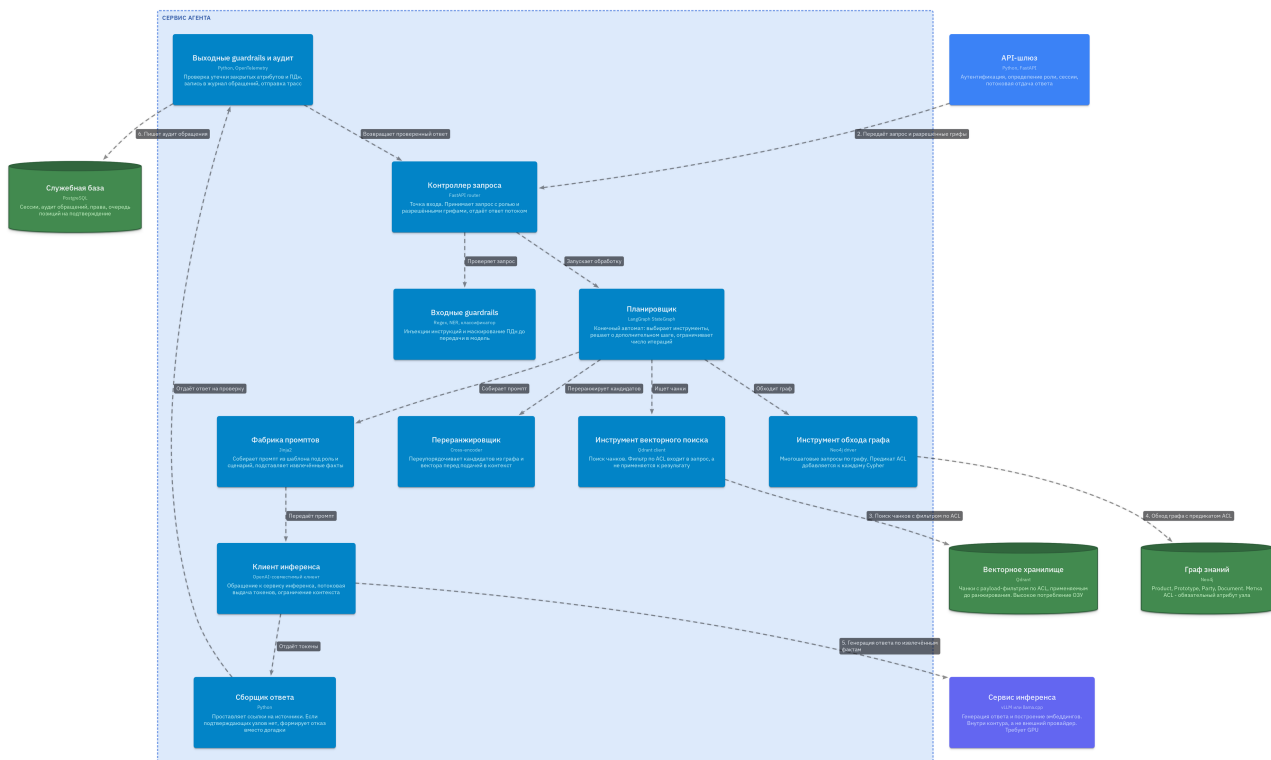


Рис. 2: Уровень 3. Компоненты сервиса агента

Компонент	Технология	Ответственность
Контроллер запроса	FastAPI router	Точка входа, приём роли и грифов, потоковая отдача
Входные guardrails	Regex, NER, классификатор	Инъекции инструкций, маскирование персональных данных
Планировщик	LangGraph StateGraph	Выбор инструментов, решение о шаге, предел итераций
Инструмент векторного поиска	Qdrant client	Поиск чанков, фильтр входит в запрос
Инструмент обхода графа	Neo4j driver	Многошаговые запросы, предикат доступа в каждом Cypher
Переранжировщик	Cross-encoder	Переупорядочивание кандидатов перед подачей в контекст
Фабрика промптов	Jinja2	Шаблон под роль и сценарий, подстановка фактов
Клиент инференса	OpenAI-совместимый	Обращение к модели, потоковая выдача, предел контекста
Сборщик ответа	Python	Простановка ссылок, отказ при отсутствии подтверждений
Выходные guardrails и аудит	Python, OpenTelemetry	Проверка утечки, журнал, трассы

Два компонента заслуживают пояснения.

Планировщик ограничивает число итераций. Без предела агент способен заикливаться на вопросе, ответа на который в графе просто нет: каждый следующий шаг будет казаться ему осмысленным. Предел превращает бесконечный поиск в честный отказ.

Сборщик ответа отвечает за отказ, а не только за сборку. Если подтверждающих узлов не нашлось, он формирует ответ “нет данных” вместо правдоподобной догадки. Это ровно то место, где лечится риск галлюцинаций при корректном контексте.

3. Диаграммы последовательности

Сценарий обработки вопроса разбит на две диаграммы. Причина не косметическая: семнадцать линий жизни в одной диаграмме нечитаемы, а граница между извлечением фактов и генерацией ответа совпадает с фазами автомата.

Под каждой диаграммой приведён перечень шагов. Диаграммы вставлены в высоком разрешении и читаются при увеличении, а таблица позволяет свериться с составом компонентов уровня 3, не прибегая к масштабированию.

Сценарий 1а. Приём запроса и извлечение фактов

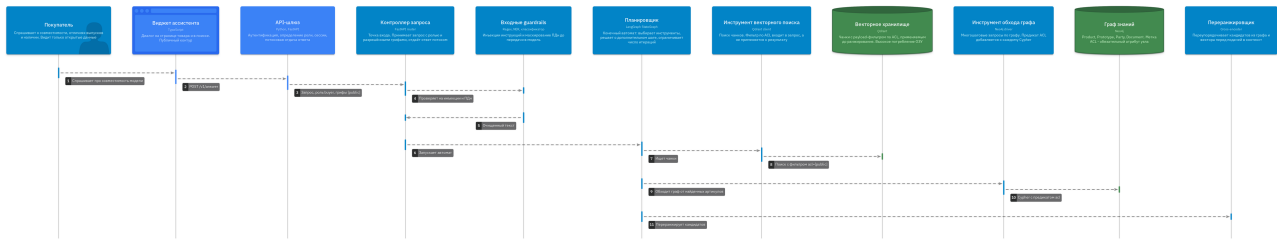


Рис. 3: Сценарий 1а

Шаг	От	К	Действие
1	Покупатель	Виджет ассистента	Спрашивает про совместимость модели
2	Виджет ассистента	API-шлюз	POST /v1/answer
3	API-шлюз	Контроллер запроса	Запрос, роль buyer, грифы [public]
4	Контроллер запроса	Входные guardrails	Проверяет на инъекции и ПДн
5	Входные guardrails	Контроллер запроса	Очищенный текст
6	Контроллер запроса	Планировщик	Запускает автомат
7	Планировщик	Инструмент векторного поиска	Ищет чанки
8	Инструмент векторного поиска	Векторное хранилище	Поиск с фильтром acl=[public]
9	Планировщик	Инструмент обхода графа	Обходит граф от найденных артикулов
10	Инструмент обхода графа	Граф знаний	Cypher с предикатом acl
11	Планировщик	Переранжировщик	Переранжирует кандидатов

Покупатель спрашивает о совместимости. Шлюз разворачивает роль в перечень разрешённых грифов и передаёт его дальше. Оба инструмента извлечения получают этот перечень и подставляют его в запрос к хранилищу.

Сценарий 1б. Генерация ответа и доставка

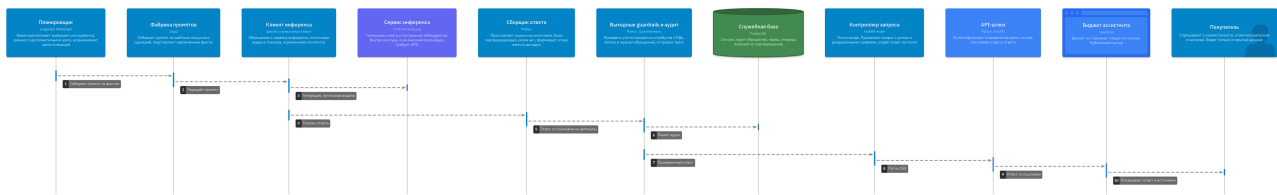


Рис. 4: Сценарий 1б

Шаг	От	К	Действие
1	Планировщик	Фабрика промптов	Собирает промпт из фактов
2	Фабрика промптов	Клиент инференса	Передаёт промпт
3	Клиент инференса	Сервис инференса	Генерация, потоковая выдача
4	Клиент инференса	Сборщик ответа	Токены ответа
5	Сборщик ответа	Выходные guardrails и аудит	Ответ со ссылками на артикулы
6	Выходные guardrails и аудит	Служебная база	Пишет аудит
7	Выходные guardrails и аудит	Контроллер запроса	Проверенный ответ
8	Контроллер запроса	API-шлюз	Поток SSE
9	API-шлюз	Виджет ассистента	Ответ со ссылками
10	Виджет ассистента	Покупатель	Показывает ответ и источники

Факты собираются в промпт, модель генерирует ответ потоком, сборщик проставляет ссылки на артикулы, выходные фильтры проверяют результат и пишут аудит.

Сценарий 2. Запрос данных вне уровня доступа

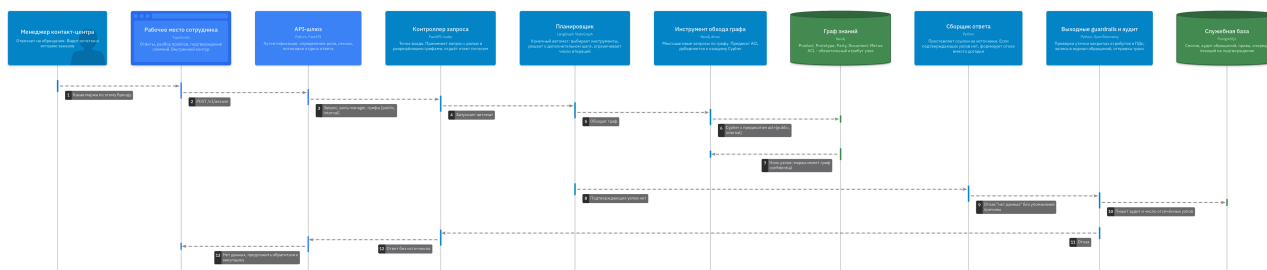


Рис. 5: Сценарий 2

Шаг	От	К	Действие
1	Менеджер контакт-центра	Рабочее место сотрудника	Какая маржа по этому бренду
2	Рабочее место сотрудника	API-шлюз	POST /v1/answer
3	API-шлюз	Контроллер запроса	Запрос, роль manager, грифы [public, internal]
4	Контроллер запроса	Планировщик	Запускает автомат
5	Планировщик	Инструмент обхода графа	Обходит граф
6	Инструмент обхода графа	Граф знаний	Cypher с предикатом acl=[public, internal]
7	Граф знаний	Инструмент обхода графа	Ноль узлов: маржа имеет гриф confidential
8	Планировщик	Сборщик ответа	Подтверждающих узлов нет
9	Сборщик ответа	Выходные guardrails и аудит	Отказ "нет данных" без упоминания причины
10	Выходные guardrails и аудит	Служебная база	Пишет аудит и число отсечённых узлов
11	Выходные guardrails и аудит	Контроллер запроса	Отказ
12	Контроллер запроса	API-шлюз	Ответ без источников
13	API-шлюз	Рабочее место сотрудника	Нет данных, предложить обратиться к закупщику

Менеджер спрашивает о маржинальности. Ключевой момент: **запрос к графу возвращает ноль узлов**, потому что узлы с маржой имеют закрытый гриф, а в предикате его нет. Отказ возникает из-за отсутствия данных, а не из-за проверки после формирования ответа.

Разница принципиальна. При постфильтрации закрытые данные успевают побывать в контексте модели, в кэше и в трассировке - и это уже утешка, даже если пользователь их не увидел.

Связность с уровнем 3

Все десять компонентов сервиса агента участвуют хотя бы в одном сценарии.

Компонент	Сценарий
Контроллер запроса	1a, 16, 2
Входные guardrails	1a
Планировщик	1a, 16, 2
Инструмент векторного поиска	1a
Инструмент обхода графа	1a, 2
Переранжировщик	1a
Фабрика промптов	16
Клиент инференса	16
Сборщик ответа	16, 2
Выходные guardrails и аудит	16, 2

4. Спецификация API

Полный файл: [backend/openapi.yaml](#), формат OpenAPI 3.1. Ниже существенные фрагменты.

Разграничение ответственности

Шлюз аутентифицирует пользователя, определяет роль и разворачивает её в перечень грифов. Дальше границы шлюза роль не уходит: сервис агента получает только перечень и подставляет его в запросы к хранилищам.

Роли во внутреннем запросе нет намеренно. Пока она передаётся, у сервиса агента два источника сведений о правах, и рано или поздно код начнёт ветвиться по роли - ровно тот отказ, ради предотвращения которого разграничение и вынесено на шлюз. Компонент не должен получать данные, которыми ему запрещено пользоваться.

Для аудита роль тоже не нужна: запись сервиса агента связывается с журналом шлюза по `session_id` и `trace_id`, а роль там уже записана в момент аутентификации.

Разделение делает права проверяемыми в отрыве от аутентификации: тест на разграничение доступа не требует поднимать вход в систему.

Запрос

```
AnswerRequest:
  type: object
  required: [question, session_id, acl]
  properties:
    question: { type: string, minLength: 3, maxLength: 2000 }
    session_id: { type: string, format: uuid }
    acl:
      type: array
      minItems: 1
      items: { type: string, enum: [public, internal, confidential] }
    stream: { type: boolean, default: false }
    max_steps: { type: integer, minimum: 1, maximum: 6, default: 3 }
```

Поле `max_steps` не декоративное: это тот самый предел итераций планировщика.

Пример запроса покупателя:

```
{
  "question": "Подойдёт ли витрина от Norev к модели 1:18 Mercedes W124",
  "session_id": "018f3a2b-7c1e-7a3d-9f42-6b0c5d8e1a77",
  "acl": ["public"],
  "stream": false
}
```

Ответ

```
{
  "answer": "Витрина Norev 1:18 подходит: высота 120 мм против 118 мм. Артикул 187001.",
  "citations": [
    { "type": "product", "id": "NOR187001", "title": "Витрина Norev 1:18" },
    { "type": "document", "id": "doc-catalog-norev-2026", "title": "Каталог Norev, с. 44" }
  ],
  "refusal": null,
  "steps": 4,
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "usage": { "prompt_tokens": 1840, "completion_tokens": 96, "latency_ms": 1420 }
}
```

Отказ приходит в том же формате, с заполненным refusal:

```
{
  "answer": null,
  "citations": [],
  "refusal": {
    "code": "no_supporting_data",
    "message": "Нет данных, подтверждающих ответ. Уточните у закупщика."
  },
  "steps": 2,
  "trace_id": "9c1e0a55b2f34e17b4de5c2a1f0d8e93"
}
```

Ответ намеренно не содержит числа отсечённых фильтром узлов. Такая величина сообщала бы, что данные существуют, но недоступны, и обещенивала бы выбор кода 200. Для разбора обращений она пишется сервисом агента прямо в журнал аудита и границу сервиса не пересекает.

Коды ответов

Код	Когда	Что делает вызывающая сторона
200	Ответ сформирован либо сформирован отказ	Показывает answer или refusal.message
400	Запрос не проходит проверку схемы	Ошибка интеграции, чинится на стороне шлюза
422	Отклонён входными фильтрами: инъекция инструкций	Показывает нейтральное сообщение, пишет инцидент
429	Превышен предел частоты	Повторяет через Retry-After
503	Инференс или хранилище недоступны	Переходит на обычный поиск по каталогу
504	Не уложились во время обработки	То же, деградация вместо ошибки

Отказ по правам возвращает 200, а не 403. Если бы отдавали 403, по коду ответа можно было бы установить сам факт существования закрытого документа: достаточно перебрать вопросы и смотреть, где меняется код. Поэтому снаружи отсутствие данных и отсутствие прав неразличимы.

Потоковый режим

При `stream: true` ответ отдаётся как Server-Sent Events: события `token` с кусками текста, затем одно событие `done` с итоговым объектом.

```
event: token  
data: {"text":"Витрина "}
```

```
event: done  
data: {"answer":"...", "citations":[...], "refusal":null, "trace_id":"4bf92f35"}
```

Источники приходят только в финальном событии: атрибуция проверяется на последнем шаге, и до его завершения список цитат не окончателен.

Документы проекта

- [Модель С4](#)
- [Спецификация API](#)
- [Бизнес-функциональные требования](#)
- [Трассировка: вопрос - риск - требование - работа - приёмка](#)
- [Реестр архитектурных решений](#)